

## Practical No. 6 — Implementation of DES Algorithm in Python

**Aim:** To implement the Data Encryption Standard (DES) algorithm in Python and understand its working through step-by-step execution.

### Pre-Requisite Knowledge

Students should know:

- ❖ Binary number representation
- ❖ XOR operation
- ❖ Python basics (loops, functions, strings)
- ❖ Concept of Feistel structure

By the end of this practical, you will:

- ❖ Convert plaintext into 64-bit binary
- ❖ Generate 16 round keys
- ❖ Perform 16 Feistel rounds
- ❖ Apply final permutation
- ❖ Generate ciphertext

### **STEP 1** — Create Basic Helper Functions (*Build the core operations used in DES.*)

```
# XOR function
def xor(a, b):
    result = ""
    for i in range(len(a)):
        if a[i] == b[i]:
            result += "0"
        else:
            result += "1"
    return result

# Permutation function
def permute(block, table):
    return ''.join([block[i - 1] for i in table])

# Left shift function
def shift_left(key_half, shifts):
    return key_half[shifts:] + key_half[:shifts]
```

You created the three main tools DES uses internally:

- Bit rearrangement
- Bit rotation
- XOR operation

Without these, DES cannot function.

**STEP 2** — Convert Plaintext and Key into Binary (*DES works on 64-bit input. Convert text to binary.*)

```
def text_to_bin(text):
    return ''.join(format(ord(c), '08b') for c in
                    text)

plaintext = "ABCDEFGH"    # Must be 8 characters
key = "ABCDEFGH"

pt_bin = text_to_bin(plaintext)
key_bin = text_to_bin(key)

print("Plaintext Binary:", pt_bin)
print("Key Binary:", key_bin)
```

*Output like this will appear*

```
Plaintext Binary:
    01000001010000100100001101000100010001010100011001000
    11101001000
Key Binary: 010000010100001001000011010001000100010101000
    1100100011101001000
=== Code Execution Successful ===
```

#### What You Achieved:

- Converted 8 characters → 64-bit binary
  - Prepared input for DES processing
- Note: If length ≠ 64 bits, DES will not work properly.**

**STEP 3** — Apply Initial Permutation (IP) (*Rearrange plaintext bits according to DES IP table.*)

```
IP = [58, 50, 42, 34, 26, 18, 10, 2,
      60, 52, 44, 36, 28, 20, 12, 4,
      62, 54, 46, 38, 30, 22, 14, 6,
      64, 56, 48, 40, 32, 24, 16, 8,
      57, 49, 41, 33, 25, 17, 9, 1,
      59, 51, 43, 35, 27, 19, 11, 3,
      61, 53, 45, 37, 29, 21, 13, 5,
      63, 55, 47, 39, 31, 23, 15, 7]

pt_ip = permute(pt_bin, IP)
print("After Initial Permutation:", pt_ip)
```

*Output like this will appear*

After Initial Permutation:

```
11111111000000000111100001010101000000000000000010000
00001100110
```

### What You Achieved:

- First official DES transformation completed
- Bits rearranged before rounds begin

### STEP 4 — Split into Left and Right Halves (*Prepare Feistel structure..*)

```
L = pt_ip[:32]
R = pt_ip[32:]

print("L0:", L)
print("R0:", R)
```

Output like this will appear

```
L0: 11111111000000000111100001010101
R0: 00000000000000000100000001100110
```

### What You Achieved:

- Created L0 and R0 (32 bits each)
- Ready for round processing

### STEP 5 — Generate 16 Round Keys (*Create 16 different 48-bit round keys.*)

a. Add PC-1 Table and apply

```
PC1 = [57,49,41,33,25,17,9,
        1,58,50,42,34,26,18,
        10,2,59,51,43,35,27,
        19,11,3,60,52,44,36,
        63,55,47,39,31,23,15,
        7,62,54,46,38,30,22,
        14,6,61,53,45,37,29,
        21,13,5,28,20,12,4]

key_pc1 = permute(key_bin, PC1)
C = key_pc1[:28]
D = key_pc1[28:]
```

b. Add Shift Schedule:

```
SHIFT_TABLE = [1, 1, 2, 2, 2, 2, 2, 2,
               1, 2, 2, 2, 2, 2, 2, 1]
```

**c. Add PC2 Table:**

```
PC2 = [14, 17, 11, 24, 1, 5,
       3, 28, 15, 6, 21, 10,
       23, 19, 12, 4, 26, 8,
       16, 7, 27, 20, 13, 2,
       41, 52, 31, 37, 47, 55,
       30, 40, 51, 45, 33, 48,
       44, 49, 39, 56, 34, 53,
       46, 42, 50, 36, 29, 32]
```

**d. Generate Keys:**

```
round_keys = []

for i in range(16):
    C = shift_left(C, SHIFT_TABLE[i])
    D = shift_left(D, SHIFT_TABLE[i])
    combined = C + D
    round_key = permute(combined, PC2)
    round_keys.append(round_key)

print("Round Key 1:", round_keys[0])
```

*Output like this will appear*

```
Round Key 1: 10100000100100100100001000010011001010101000
             0010
```

**What You Achieved:**

- Successfully generated 16 different keys
- Key scheduling phase completed

**STEP 6 — Perform 16 Feistel Rounds** (*Perform 16 Feistel Rounds.*)

*a. Add Expansion Table:*

```
E = [32,1,2,3,4,5,
      4,5,6,7,8,9,
      8,9,10,11,12,13,
      12,13,14,15,16,17,
      16,17,18,19,20,21,
      20,21,22,23,24,25,
      24,25,26,27,28,29,
      28,29,30,31,32,1]
```

b. Write Round Loop:

```
for i in range(16):
    R_expanded = permute(R, E)
    xor_result = xor(R_expanded, round_keys[i])

    R_new = xor(L, xor_result[:32])    #
        simplified (without S-box for now)
    L = R
    R = R_new

    print("Round", i+1)
    print("L:", L)
    print("R:", R)
```

*Output like this will appear*

```
Round 1
L: 00000000000000001000000001100110
R: 01011111100100100011101100000110
Round 2
L: 01011111100100100011101100000110
R: 1000111111011100111011001101001
```

•  
•  
•  
•

```

Round 15
L: 10011010110111010111000111110111
R: 01101101100001111100011010101001
Round 16
L: 01101101100001111100011010101001
R: 1000111111100110101110010110100

```

**What You Achieved:**

- Implemented Feistel structure
- Observed how L and R change each round

**STEP 7 — Final Permutation** (*Generate final ciphertext.*)

Add FP table and generate cipher text

```

FP = [40,8,48,16,56,24,64,32,
      39,7,47,15,55,23,63,31,
      38,6,46,14,54,22,62,30,
      37,5,45,13,53,21,61,29,
      36,4,44,12,52,20,60,28,
      35,3,43,11,51,19,59,27,
      34,2,42,10,50,18,58,26,
      33,1,41,9,49,17,57,25]

combined = R + L
cipher_text = permute(combined, FP)

print("Final Ciphertext (Binary):", cipher_text)

```

*Output like this will appear*

```

L0: 11111111000000000111100001010101
R0: 00000000000000000100000001100110

```

**What You Achieved:**

- Created L0 and R0 (32 bits each)
- Ready for round processing

**STEP 8— Convert Binary to ASCII Characters** *(to get the ciphertext)*

```
def bin_to_text(binary_string):
    text = ""
    for i in range(0, len(binary_string), 8):
        byte = binary_string[i:i+8]
        decimal = int(byte, 2)
        text += chr(decimal)
    return text

cipher_text_readable = bin_to_text(cipher_text)
print("Ciphertext (Readable Form):",
      cipher_text_readable)
```

Output like this will appear

```
Ciphertext (Readable Form): òxíÆ...{
```

**Why Is It Looking Weird?**

Because:

- Every 8 bits = 1 ASCII character
- But encrypted binary is random
- Random 8-bit values = random ASCII characters

**Question 1 — Implement DES Encryption Step-by-Step**

Write a Python program to implement the DES encryption algorithm using:

- Initial Permutation
- Key Generation (16 round keys)
- 16 Feistel Rounds
- Final Permutation

**CODE:**

```
IP = [58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,
      62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
      57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,
      61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7]
```

```
FP = [40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,
      38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,
      36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,
      34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25]
```

E = [32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,  
12,13,12,13,14,15,16,17,16,17,18,19,  
20,21,20,21,22,23,24,25,24,25,26,27,  
28,29,28,29,30,31,32,1]

P = [16,7,20,21,29,12,28,17,1,15,23,26,  
5,18,31,10,2,8,24,14,32,27,3,9,  
19,13,30,6,22,11,4,25]

PC1 = [57,49,41,33,25,17,9,1,58,50,42,34,26,18,  
10,2,59,51,43,35,27,19,11,3,60,52,44,36,  
63,55,47,39,31,23,15,7,62,54,46,38,30,22,  
14,6,61,53,45,37,29,21,13,5,28,20,12,4]

PC2 = [14,17,11,24,1,5,3,28,15,6,21,10,  
23,19,12,4,26,8,16,7,27,20,13,2,  
41,52,31,37,47,55,30,40,51,45,33,48,  
44,49,39,56,34,53,46,42,50,36,29,32]

SHIFT = [1,1,2,2,2,2,2,2,1,2,2,2,2,2,1]

S\_BOX = [  
[[14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7],  
[0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8],  
[4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0],  
[15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13]],  
[[15,1,8,14,6,11,3,4,9,7,2,13,12,0,5,10],  
[3,13,4,7,15,2,8,14,12,0,1,10,6,9,11,5],  
[0,14,7,11,10,4,13,1,5,8,12,6,9,3,2,15],  
[13,8,10,1,3,15,4,2,11,6,7,12,0,5,14,9]],  
[[10,0,9,14,6,3,15,5,1,13,12,7,11,4,2,8],  
[13,7,0,9,3,4,6,10,2,8,5,14,12,11,15,1],  
[13,6,4,9,8,15,3,0,11,1,2,12,5,10,14,7],  
[1,10,13,0,6,9,8,7,4,15,14,3,11,5,2,12]],  
[[7,13,14,3,0,6,9,10,1,2,8,5,11,12,4,15],  
[13,8,11,5,6,15,0,3,4,7,2,12,1,10,14,9],  
[10,6,9,0,12,11,7,13,15,1,3,14,5,2,8,4],  
[3,15,0,6,10,1,13,8,9,4,5,11,12,7,2,14]],  
[[2,12,4,1,7,10,11,6,8,5,3,15,13,0,14,9],  
[14,11,2,12,4,7,13,1,5,0,15,10,3,9,8,6],  
[4,2,1,11,10,13,7,8,15,9,12,5,6,3,0,14],  
[11,8,12,7,1,14,2,13,6,15,0,9,10,4,5,3]],  
[[12,1,10,15,9,2,6,8,0,13,3,4,14,7,5,11],  
[10,15,4,2,7,12,9,5,6,1,13,14,0,11,3,8],  
[9,14,15,5,2,8,12,3,7,0,4,10,1,13,11,6],  
[4,3,2,12,9,5,15,10,11,14,1,7,6,0,8,13]],  
[[4,11,2,14,15,0,8,13,3,12,9,7,5,10,6,1],



```
[13,0,11,7,4,9,1,10,14,3,5,12,2,15,8,6],
[1,4,11,13,12,3,7,14,10,15,6,8,0,5,9,2],
[6,11,13,8,1,4,10,7,9,5,0,15,14,2,3,12]],
[[13,2,8,4,6,15,11,1,10,9,3,14,5,0,12,7],
[1,15,13,8,10,3,7,4,12,5,6,11,0,14,9,2],
[7,11,4,1,9,12,14,2,0,6,10,13,15,3,5,8],
[2,1,14,7,4,10,8,13,15,12,9,0,3,5,6,11]]
]
```

```
def permute(block, table):
    return ''.join(block[i-1] for i in table)

def shift_left(k, n):
    return k[n:] + k[:n]

def xor(a, b):
    return ''.join('0' if i == j else '1' for i, j in zip(a, b))

def hex_to_bin(h):
    return bin(int(h, 16))[2:].zfill(64)

def bin_to_hex(b):
    return hex(int(b, 2))[2:].upper().zfill(16)

def generate_keys(key):
    key = permute(key, PC1)
    left, right = key[:28], key[28:]
    keys = []
    for i in range(16):
        left = shift_left(left, SHIFT[i])
        right = shift_left(right, SHIFT[i])
        keys.append(permute(left+right, PC2))
    return keys

def sbx_substitution(expanded):
    result = ""
    for i in range(8):
        block = expanded[i*6:(i+1)*6]
        row = int(block[0]+block[5], 2)
        col = int(block[1:5], 2)
        result += format(S_BOX[i][row][col], '04b')
    return result

def feistel(right, key):
    expanded = permute(right, E)
    xored = xor(expanded, key)
    sbx = sbx_substitution(xored)
```

```
def des_encrypt(pt, key):
    pt = permute(pt, IP)
    left, right = pt[:32], pt[32:]
    keys = generate_keys(key)
    for i in range(16):
        temp = right
        right = xor(left, feistel(right, keys[i]))
        left = temp
    return permute(right+left, FP)

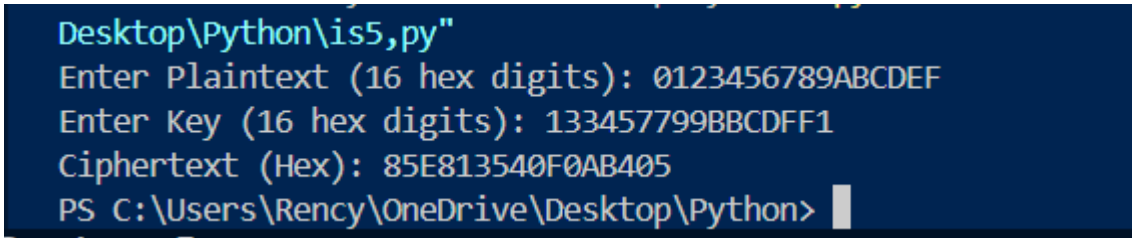
pt_hex = input("Enter Plaintext (16 hex digits): ")
key_hex = input("Enter Key (16 hex digits): ")

pt = hex_to_bin(pt_hex)
key = hex_to_bin(key_hex)

cipher_bin = des_encrypt(pt, key)
cipher_hex = bin_to_hex(cipher_bin)

print("Ciphertext (Hex):", cipher_hex)
```

### OUTPUT:



```
Desktop\Python\is5.py
Enter Plaintext (16 hex digits): 0123456789ABCDEF
Enter Key (16 hex digits): 133457799BBCDF1
Ciphertext (Hex): 85E813540F0AB405
PS C:\Users\Rency\OneDrive\Desktop\Python>
```

### Question 2 — Convert Ciphertext to Hexadecimal (Compulsory)

After generating the final 64-bit binary ciphertext, modify your program so that it displays the ciphertext in **hexadecimal format**.

### CODE:

```
IP = [58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,
      62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
      57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,
      61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7]

FP = [40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,
      38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,
      36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,
      34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25]

E = [32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,
```

12,13,12,13,14,15,16,17,16,17,18,19,  
20,21,20,21,22,23,24,25,24,25,26,27,  
28,29,28,29,30,31,32,1]

P = [16,7,20,21,29,12,28,17,1,15,23,26,  
5,18,31,10,2,8,24,14,32,27,3,9,  
19,13,30,6,22,11,4,25]

PC1 = [57,49,41,33,25,17,9,1,58,50,42,34,26,18,  
10,2,59,51,43,35,27,19,11,3,60,52,44,36,  
63,55,47,39,31,23,15,7,62,54,46,38,30,22,  
14,6,61,53,45,37,29,21,13,5,28,20,12,4]

PC2 = [14,17,11,24,1,5,3,28,15,6,21,10,  
23,19,12,4,26,8,16,7,27,20,13,2,  
41,52,31,37,47,55,30,40,51,45,33,48,  
44,49,39,56,34,53,46,42,50,36,29,32]

SHIFT = [1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1]

S\_BOX = [[[14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7],  
[0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8],  
[4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0],  
[15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13]]] \* 8

def permute(block, table):

    return ''.join(block[i-1] for i in table)

def shift\_left(k, n):

    return k[n:] + k[:n]

def xor(a, b):

    return ''.join('0' if i == j else '1' for i, j in zip(a, b))

def hex\_to\_bin(h):

    return bin(int(h, 16))[2:].zfill(64)

def bin\_to\_hex(b):

    return hex(int(b, 2))[2:].upper().zfill(16)

def generate\_keys(key):

    key = permute(key, PC1)

    left, right = key[:28], key[28:]

```
keys = []
for i in range(16):
    left = shift_left(left, SHIFT[i])
    right = shift_left(right, SHIFT[i])
    keys.append(permute(left+right, PC2))
return keys

def sbbox_substitution(expanded):
    result = ""
    for i in range(8):
        block = expanded[i*6:(i+1)*6]
        row = int(block[0]+block[5], 2)
        col = int(block[1:5], 2)
        val = S_BOX[i][row][col]
        result += format(val, '04b')
    return result

def feistel(right, key):
    expanded = permute(right, E)
    xored = xor(expanded, key)
    sbbox = sbbox_substitution(xored)
    return permute(sbbox, P)

def des_encrypt(pt, key):
    pt = permute(pt, IP)
    left, right = pt[:32], pt[32:]
    keys = generate_keys(key)
    for i in range(16):
        temp = right
        right = xor(left, feistel(right, keys[i]))
        left = temp
    return permute(right+left, FP)

pt_hex = input("Enter Plaintext (16 hex digits): ")
key_hex = input("Enter Key (16 hex digits): ")

pt = hex_to_bin(pt_hex)
key = hex_to_bin(key_hex)

cipher_bin = des_encrypt(pt, key)
cipher_hex = bin_to_hex(cipher_bin)

print("Ciphertext (Binary):", cipher_bin)
```

```
print("Ciphertext (Hex):", cipher_hex)
```

## OUTPUT:

```
Desktop\Python\is5.py"
Enter Plaintext (16 hex digits): 0123456789ABCDEF
Enter Key (16 hex digits): 133457799BBCDFF1
Ciphertext (Binary): 10111110101101010101010101111100000100110100111010100110
1001110
Ciphertext (Hex): BEB5557E09A7534E
PS C:\Users\Rency\OneDrive\Desktop\Python>
```

### Question 3 — Compare 1 Round, 3 Rounds and 16 Rounds

Modify your DES loop to run with:

1. Only 1 round
2. Only 3 rounds
3. Full 16 rounds

## CODE:

```
IP = [58,50,42,34,26,18,10,2,60,52,44,36,28,20,12,4,
      62,54,46,38,30,22,14,6,64,56,48,40,32,24,16,8,
      57,49,41,33,25,17,9,1,59,51,43,35,27,19,11,3,
      61,53,45,37,29,21,13,5,63,55,47,39,31,23,15,7]
```

```
FP = [40,8,48,16,56,24,64,32,39,7,47,15,55,23,63,31,
      38,6,46,14,54,22,62,30,37,5,45,13,53,21,61,29,
      36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,
      34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25]
```

```
E = [32,1,2,3,4,5,4,5,6,7,8,9,8,9,10,11,
      12,13,12,13,14,15,16,17,16,17,18,19,
      20,21,20,21,22,23,24,25,24,25,26,27,
      28,29,28,29,30,31,32,1]
```

```
P = [16,7,20,21,29,12,28,17,1,15,23,26,
      5,18,31,10,2,8,24,14,32,27,3,9,
      19,13,30,6,22,11,4,25]
```

```
PC1 = [57,49,41,33,25,17,9,1,58,50,42,34,26,18,
        10,2,59,51,43,35,27,19,11,3,60,52,44,36,
        63,55,47,39,31,23,15,7,62,54,46,38,30,22,
        14,6,61,53,45,37,29,21,13,5,28,20,12,4]
```

```
PC2 = [14,17,11,24,1,5,3,28,15,6,21,10,  
       23,19,12,4,26,8,16,7,27,20,13,2,  
       41,52,31,37,47,55,30,40,51,45,33,48,  
       44,49,39,56,34,53,46,42,50,36,29,32]
```

```
SHIFT = [1,1,2,2,2,2,2,2,1,2,2,2,2,2,1]
```

```
S_BOX = [[[14,4,13,1,2,15,11,8,3,10,6,12,5,9,0,7],  
          [0,15,7,4,14,2,13,1,10,6,12,11,9,5,3,8],  
          [4,1,14,8,13,6,2,11,15,12,9,7,3,10,5,0],  
          [15,12,8,2,4,9,1,7,5,11,3,14,10,0,6,13]]] * 8
```

```
def permute(block, table):  
    return ''.join(block[i-1] for i in table)
```

```
def shift_left(k, n):  
    return k[n:] + k[:n]
```

```
def xor(a, b):  
    return ''.join('0' if i == j else '1' for i, j in zip(a, b))
```

```
def hex_to_bin(h):  
    return bin(int(h, 16))[2:].zfill(64)
```

```
def bin_to_hex(b):  
    return hex(int(b, 2))[2:].upper().zfill(16)
```

```
def generate_keys(key):  
    key = permute(key, PC1)  
    left, right = key[:28], key[28:]  
    keys = []  
    for i in range(16):  
        left = shift_left(left, SHIFT[i])  
        right = shift_left(right, SHIFT[i])  
        keys.append(permute(left+right, PC2))  
    return keys
```

```
def sbbox_substitution(expanded):  
    result = ""  
    for i in range(8):  
        block = expanded[i*6:(i+1)*6]  
        row = int(block[0]+block[5], 2)  
        col = int(block[1:5], 2)
```

```
    val = S_BOX[i][row][col]
    result += format(val, '04b')
return result

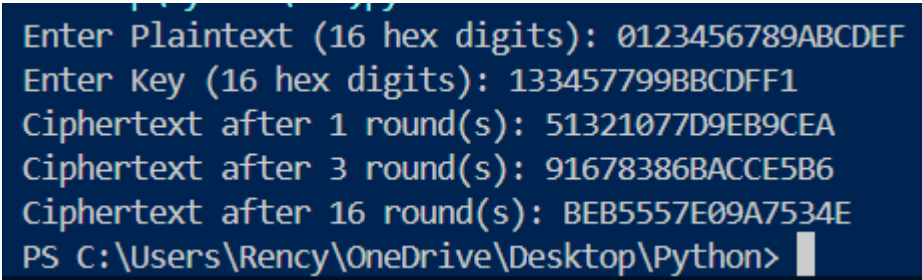
def feistel(right, key):
    expanded = permute(right, E)
    xored = xor(expanded, key)
    sbox = sbox_substitution(xored)
    return permute(sbox, P)

def des_encrypt(pt, key, rounds):
    pt = permute(pt, IP)
    left, right = pt[:32], pt[32:]
    keys = generate_keys(key)
    for i in range(rounds):
        temp = right
        right = xor(left, feistel(right, keys[i]))
        left = temp
    return permute(right+left, FP)

pt_hex = input("Enter Plaintext (16 hex digits): ")
key_hex = input("Enter Key (16 hex digits): ")

pt = hex_to_bin(pt_hex)
key = hex_to_bin(key_hex)

for r in [1,3,16]:
    cipher_bin = des_encrypt(pt, key, r)
    cipher_hex = bin_to_hex(cipher_bin)
    print(f"Ciphertext after {r} round(s):", cipher_hex)
```

**OUTPUT:**

```
Enter Plaintext (16 hex digits): 0123456789ABCDEF
Enter Key (16 hex digits): 133457799BBCDFF1
Ciphertext after 1 round(s): 51321077D9EB9CEA
Ciphertext after 3 round(s): 91678386BACCE5B6
Ciphertext after 16 round(s): BEB5557E09A7534E
PS C:\Users\Rency\OneDrive\Desktop\Python>
```